

Project Report

for

Automatic Discovery of Metamorphic Relations

<https://github.com/kubaskibicki/CS453.git>

Yeskendir Assankul

Ada Jacyna

Jakub Skibicki

Hugo Tarczyński

June 10, 2026

1. Introduction	1
2. Problem	2
2.1. Testing obstacles	2
2.1.1. The Oracle	2
2.2. Metamorphic Testing	3
2.2.1. Examples of metamorphing relations	3
2.2.2. Discovery problem	3
3. Solution of the problem	3
3.1. The Relation Template	3
3.2. Description of the Tool	4
3.2.1. Function reconnaissance with an LLM	4
3.2.2. Turning a profile into search bounds	4
3.2.3. The PSO search and fitness function	5
3.2.4. Collecting and deduplicating relations	5
3.2.5. Scoring relations against mutants	5
3.2.6. Comparing against MRI and AutoMR on Apache Commons Math 2.2	6
4. Tool evaluation	7
4.1. Results evaluation	7
4.2. Comparison to other tools	8
4.3 Future work	9

1. Introduction

Testing numerical software is difficult because the expected output is often hard to determine automatically. Mathematical functions may accept infinitely many real-valued inputs, operate under domain restrictions, and produce results affected by floating-point error. As a result, ordinary unit tests based on fixed input-output pairs are often insufficient. This project focuses on the automatic discovery of metamorphic relations for mathematical functions. A metamorphic relation describes how the outputs of a function should change when its inputs are transformed.

For example, instead of checking the exact value of $\sin(x)$, a tester can check whether $\sin(x) + \sin(-x) = 0$. The goal of the project is to reduce the manual effort required to apply metamorphic testing. Our tool searches for relations automatically by combining a local language model, which provides high-level hints about a function's behavior, with Particle Swarm Optimization, which searches for concrete numerical parameters. The tool is evaluated on eight functions: *abs*, *asinh*, *atan*, *cos*, *log1p*, *log10*, *sin*, and *tan*.

2. Problem

The main problem addressed in this project is the automatic discovery of useful metamorphic relations. Metamorphic testing is effective when good relations are known, but finding them usually requires mathematical knowledge and manual work. This does not scale well when many functions have to be tested.

2.1. Testing obstacles

Numerical software is challenging to test for several reasons. The input space is usually continuous, so exhaustive testing is impossible. Many functions also have restricted or unstable domains. For example, logarithmic functions cannot be evaluated on invalid inputs, and tangent becomes unstable near its asymptotes. Floating-point arithmetic adds another difficulty. Correct implementations may produce slightly different results because of rounding, so tests usually require tolerances instead of exact equality. If the tolerance is too strict, correct outputs may be rejected - if it is too loose, real faults may remain undetected. Finally, numerical faults can be subtle. A wrong constant, changed sign, shifted input, or modified operator may still produce plausible values. Simple example-based tests may therefore miss errors that violate deeper mathematical properties.

2.1.1. The Oracle

A test oracle decides whether a program's output is correct. In standard unit testing, the oracle is often a known expected value. This works when the correct answer is easy to compute. For mathematical software, such an oracle is often unavailable. Computing the exact expected result may be as difficult as implementing the tested function itself. Comparing it against another library can help, but this is not always possible, and it cannot be used when the library itself is the subject of testing. Metamorphic testing avoids the need for exact expected outputs. Instead of checking whether $P(x)$ is correct

directly, it checks whether several executions of P satisfy a required relation. If the relation is violated, the implementation is likely faulty.

2.2. Metamorphic Testing

Metamorphic testing generates follow-up test cases from existing inputs. The original input x is transformed into another input, such as $-x$, $x + \pi$, or kx , and the outputs are compared according to a mathematical property. This makes metamorphic testing useful for programs without a traditional oracle. The tester does not need to know the exact value returned by the function. It is enough to know how outputs should relate to each other. In this project, metamorphic testing is applied to mathematical functions because many of them have symmetry, periodicity, or scaling properties.

2.2.1. Examples of metamorphing relations

For an odd function such as sine, changing the sign of the input changes the sign of the output:

$$\sin(-x) = -\sin(x)$$

This can be tested as:

$$\sin(x) + \sin(-x) = 0$$

For an even function such as cosine, changing the sign of the input does not change the output:

$$\cos(x) - \cos(-x) = 0$$

Periodic functions give shift-based relations. For example:

$$\sin(x + 2\pi) - \sin(x) = 0$$

$$\cos(x + 2\pi) - \cos(x) = 0$$

$$\tan(x + \pi) - \tan(x) = 0$$

2.2.2. Discovery problem

The discovery problem is to find such relations automatically for a given function P . In this project, the searched relation must be accurate, non-trivial, and useful for fault detection. Accuracy means that the relation should hold not only on the sample inputs used during search, but also on fresh validation inputs. Non-triviality means that the relation must not be satisfied only because all coefficients are zero or because the function is compared with itself. Fault detection means that the relation should be able to distinguish the correct function from mutated, faulty versions. This is difficult because the search space is continuous and contains many misleading solutions. Different functions also require different relation types: periodic functions need shift relations, odd functions need sign-change relations, and logarithmic functions may need scaling relations.

3. Solution of the problem

3.1. The Relation Template

A metamorphic relation links the outputs of a function on two related inputs, so the tester does not need to know the correct answer for either one. The classic example is $\sin(x) = -\sin(-x)$: you can check it without knowing the value of $\sin(x)$.

The hard part of metamorphic testing is coming up with those relations in the first place. For a single function this is a manual job that needs someone who understands the math. Our tool replaces that step with a search. We fix one template and let everything else be a parameter:

$$c1 \cdot P(x) + c2 \cdot P(a \cdot x + b) + d = 0$$

for all x in the domain

Here P is the function under test and $c1, c2, a, b, d$ are five real numbers. Different settings recover familiar relations. The odd-function rule $\sin(x) + \sin(-x) = 0$ is:

$$c1=1, c2=1, a=-1, b=0, d=0.$$

The periodic shift $\sin(x) - \sin(x + 2\pi) = 0$ is:

$$c1=1, c2=-1, a=1, b=2\pi, d=0.$$

Because the whole template collapses to five numbers, finding a relation becomes a numerical optimization problem: choose the five values that make the left side as close to zero as possible across many sample inputs.

3.2. Description of the Tool

The tool runs each function through five stages: an LLM reconnaissance pass, a translation from the function's profile into search bounds, a PSO¹ search that repeats with several restarts, a deduplication pass, and a mutation-based scoring step. The command-line entry point (`amr/cli.py`) ties these together and writes one JSON result per function plus a `summary.csv`.

3.2.1. Function reconnaissance with an LLM

Before searching, the tool asks a local language model what kind of function it is dealing with (`amr/recon.py`). We send *qwen2.5:7b-instruct*, served through *Ollama*, a short prompt that demands a JSON answer with six fields: symmetry (even, odd, or none), whether the function is periodic, its period, monotonicity, a safe domain, and an output range. The model never sees the test inputs and never produces a relation. It only describes properties a competent student would know, such as "*sin is odd and periodic with period 2pi.*" We treat its answer as a hint, not as ground truth, which is why the next stages stay robust when the hint is wrong. Each answer is cached on disk so repeated runs do not call the model again, and if the call fails the tool logs a warning and falls back to wide default bounds.

¹ PSO - Particle Swarm Optimization

3.2.2. Turning a profile into search bounds

The model's profile is converted into bounds for the five parameters (`amr/bounds.py`). The default box is deliberately loose: c_1, c_2, a, d range over $[-2, 2]$ and b over $[-10, 10]$. The profile then pins down whatever it can. If the function is odd, we fix $a = -1$ and $b = 0$, which steers the search straight at the $P(x) + P(-x) = 0$ family. If it is periodic with period p , we fix $a = 1$ and limit b to $[-p, p]$, which is where shift relations live. One case is left alone on purpose. Even symmetry gives $P(x) - P(-x) = 0$, but that holds for every mutant too, so it kills nothing and we do not exploit it. This is the kind of distinction that makes the LLM hint useful: it tells the search where the real relations are without wasting effort on relations that cannot detect a fault.

3.2.3. The PSO search and fitness function

The search itself is a standard particle swarm (`amr/pso.py`). Fifty particles start uniformly inside the bounds and move under the usual inertia, personal-best, and global-best terms, with the coefficients set to 0.7, 1.5, and 1.5. Each particle is clamped back inside the bounds after every step. The run stops early once the global best has not improved by more than $1e-8$ for 30 iterations, and otherwise after 350 iterations. The fitness function decides what "a good relation" means (`amr/mr.py`). Its core is the mean squared value of the template's left side over 200 sample inputs, so a particle scores well when the relation holds. The raw error alone is not enough, because the all-zero setting $c_1=c_2=d=0$ satisfies the template and means nothing. Three penalties rule out those degenerate answers:

- A trivial relation where the coefficient mass $|c_1| + |c_2|$ falls below 0.5 is pushed away from the search.
- A relation where the transformed input $a*x + b$ barely differs from x is penalized, since comparing a function to itself proves nothing.
- A relation with a near zero is penalized, because that turns the second term into a constant.

A separate validation step (`validate` in `amr/mr.py`) re-checks each candidate on a fresh set of inputs drawn from a different seed. A candidate is accepted only if its worst-case residual stays under the tolerance of $1e-3$ and it passes the same non-triviality checks. Using held-out inputs here is what stops the tool from reporting relations that only happened to fit the points the swarm trained on.

3.2.4. Collecting and deduplicating relations

A single search returns one relation, and one relation rarely catches every fault. So the tool runs eight restarts per function (`run_function` in `amr/run.py`), each with its own seed. The restarts alternate between two bound sets. Even-numbered restarts use the tight LLM-guided bounds, which land on the canonical relation quickly. Odd-numbered restarts use the wide default bounds, which wander further and tend to find odder, more varied relations. Together they cover more of the fault space than either would alone. Eight restarts produce overlapping answers, since the canonical relation shows up again and again under slightly different numbers. The deduplication step normalizes each relation by dividing the amplitude coefficients by the first non-zero one and

rounding, then keeps one representative per distinct fingerprint. For sin, this is how a run that found the 2π shift in three different restarts reports it once.

3.2.5. Scoring relations against mutants

The last stage measures whether the discovered relations are any good at finding bugs (`amr/metrics.py`). For each function we generate 78 mutants, 625 in total to match the AutoMR benchmark count (`scripts/build_mutants.py`). A mutant is a deliberately broken version of the function, built from one of six operators: adding or multiplying a constant, shifting or scaling the input, replacing the output with a constant, or negating it (`amr/mutants.py`). A relation kills a mutant when the relation, which holds for the real function, is violated by the mutant beyond the tolerance. We report the kill rate, the fraction of mutants caught by at least one of a function's relations. On the eight benchmark functions the kill rates ranged from 32% on `tan`, whose narrow usable domain leaves little room for shift relations, to 79.5% on `log1p`. Each run also records how many relations survived validation and how many PSO iterations it took, which the tool writes to the per-function JSON and the summary file.

3.2.6. Comparing against MRI and AutoMR on Apache Commons Math 2.2

The scoring above uses our own mutants of numpy functions. That tells us whether the relations catch faults, but it does not put the tool next to the published baselines. MRI and AutoMR both report kill rates on a fixed benchmark: 625 mutants of eight functions from Apache Commons Math 2.2. To compare on the same ground, we rebuilt the scoring stage on those subjects. The 625 mutants themselves are not available anywhere. They are mutated copies of the Java source, and the AutoMR repository ships only the relations it inferred, not the faulty programs. So we reproduce the benchmark instead of downloading it: the same eight functions, mutated the same way, scored the same way.

The eight functions live in `"org.apache.commons.math.util.FastMath"` in the 2.2 release (the jar sits in `"commons-math-2.2/"`). We ported them to Python in `"amr/commons_math.py"` and kept the real algorithm and constants: the Cody-Waite range reduction, the $1/8$ -radian sine, cosine and tangent tables, the Remez polynomials for the small-angle correction, and the series for `asinh` and `log1p`. The one thing we dropped is FastMath's split-double bookkeeping, which exists to recover the last bit of accuracy and has nothing to do with the code a mutation would change. The port agrees with numpy to about $1e-16$ on the trigonometric functions and $1e-11$ on the logarithms, so it is a correct oracle to test against.

`"amr/commons_mutants.py"` makes 625 mutants of those ports. Each mutant changes one token in a function's code: an arithmetic operator (`"+"` becomes `"-"`, `"*"` becomes `"/"`), a comparison (`"<"` becomes `"<="`), or a numeric constant. These are the same operator kinds a Java mutation tool applies, which is how the original 625 were produced. Each mutant is compiled back into a callable. One detail matters here. A mutated loop condition can stop a function from terminating, so we rewrite every while

loop into a bounded one. A mutant that would otherwise spin forever now raises instead, and a mutant that crashes on an input counts as caught.

A flat tolerance does not work well on this benchmark. Our relations hold on the true function only up to some small residual, and that residual is not the same for every relation. Most sit near $1e-5$, but a few of the log relations sit near $1e-3$. Judged against a single $1e-3$ line, those loose relations look like they fire on the correct function, which is a false alarm. So we judge each relation against its own noise floor. We measure the relation’s residual on the correct function, then count a mutant as caught only when its residual is at least ten times larger. A relation can never flag the function it was fit to, since its own residual is below its own threshold by construction, so the false-detection count is zero. Tight relations still catch small faults, and the loose log relations become usable instead of discarded. Inputs where the function is undefined are dropped, and $\log_{10}$ and $\log_{1p}$ clamp the shifted input the same way the search does.

4. Tool evaluation

In order to evaluate the performance of our tool, we divided the testing into a two-part analysis. First, we tested our tool against a custom set of mutated NumPy function, then against the industry standard Apache Commons Math 2.2 benchmark, for comparison with existing industry baselines. This source provided us with 8 functions to test our tool against: *abs*, *asinh*, *atan*, *cos*, *log1p*, *log10*, *sin*, *tan*.

4.1. Results evaluation

4.1.1. Metrics obtained

The tool was tested based on two major quality requirements: efficiency and effectiveness. The *efficiency* was evaluated using the metrics of *iterations* of the tool (before finishing the operation) and *runtime*. The *effectiveness*, in turn, was rated based on the *kill rate*: representation of the count of the mutants caught by at least one MR, and *precision*, which serves as moderation of how accurate the generated math solutions are. It was measured using the false detections count and because we used a dynamic tolerance threshold, rather than a flat baseline, the false-positive rate is, by design, zero.

4.1.2. Data and results

When evaluated against our initial NumPy mutant baseline (comprising 78 mutants per function), the relations generated by our tool demonstrated strong fault detection, though the success varied across different mathematical domains. The overall kill rates reached a high of 79.49%²[table 1] on *log1p* and dropped to a low of 32.05% on *tan*, whose narrow usable domain leaves little room for shift relations. In terms of efficiency, the average PSO iterations ranged from 103.6 loops up to 270.6 loops, with per-function average runtimes scaling tightly between 5.88 and 11.46 seconds.

² Mutants killed per mutants found

Evaluated function	Kill rate [%]	MR count	MR precision [%]	avg PSO iterations	runtime [s]
abs	50.0	13	100.0	196.438	11.461
asinh	52.56	1	9.091	106.936	5.88
atan	42.31	1	10.0	103.625	7.769
cos	71.79	8	50.0	103.688	6.85
log1p	79.49	3	100.0	270.563	11.014
log10	66.67	2	100.0	196.125	9.909
sin	71.79	6	42.857	112.75	7.13
tan	32.05	2	25.0	165.375	10.053

Table 1. Our tool - summary of results against the NumPy generated mutants.

4.2. Comparison to other tools

To position our tool against the existing research, we compared our performance against the already mentioned two known industry solutions: MRI and AutoMR. Due to the unavailability of the original Java-based 625 mutants from the MRI paper, we evaluated all tools on a faithful Python port using the same mutation strategies, albeit not the exact recreations.

On this aggregate benchmark [Table 2.], our tool achieved a total kill rate of 22.7% (142 out of 625 mutants), outperforming the MRI baseline which achieved 15.04% (94 out of 625 mutants). While AutoMR serves as an upper bound at 59.84% (374 out of 625 mutants), our tool achieved its results with massive efficiency gains. Our tool required only 8 restarts of 157 average iterations with a 50-particle swarm, yielding a blistering runtime of just 1.9 seconds per MR, compared to MRI’s 3.9 seconds and AutoMR’s 4.7 seconds on a much heavier 500x350 iteration setup.

Metric	MRI (baseline)	AutoMR (upper bound)	Ours (LLM-guided PSO)
Mutants killed	94/625	374/625	142/625
Kill rate	15.04%	59.84%	22.7%
False detections	0	0	0
Runtime per MR (s)	3.9	4.7	1.9

PSO runs x iters	500x350	500x350	16x157 (50 particles)
------------------	---------	---------	-----------------------

Table 2. Overall statistics tool comparison.

Breaking down the performance by individual functions, our tool successfully beat the AutoMR baseline on *log10* (41.2% vs 34.5%) and performed competitively on *sin* (50.8% vs 52.9%). It faced limitations on complex logarithmic and inverse trigonometric behaviors like *asinh* and *atan*, where the tight dynamic noise thresholds restricted the relation pool. However, the zero false-detection rate across all configurations proves that our tool prioritizes highly precise, accurate relations at a fraction of the computational runtime of prior methods.

Function	AutoMR killed/faults	AutoMR kill %	Ours kill %*	Ours #MR
abs	10/14	71.4	50.0	13
asinh	70/297	23.6	1.6	1
atan	96/188	51.1	16.1	1
cos	16/38	42.1	25.0	8
log1p	92/230	40.0	27.4	3
log10	40/116	34.5	41.2	2
sin	18/34	52.9	50.8	6
tan	32/36	88.9	54.2	2

*Ours kills source-level mutants of the Commons Math 2.2 port, built the same way as AutoMR's mutants but not the identical Java set; AutoMR's per-function denominators are its own.

Table 3. Per function tool comparison between AutoMR and Our tool.

4.3 Future work

Our tool successfully bested the original industry solution, proving we have completed our goal. However, it still significantly fell behind the AutoMR in terms of effectiveness. Future improvements to the tool would encompass creating a better LLM, trained on specific data and relations, making it more accurate in its computations. In an optimal situation, we would also find even better parameters for the PSO algorithm, hopefully increasing both effectiveness and efficiency. In the end, our goal was not to recreate the AutoMR but to make a different tool that would improve something on the existing one. That suggests that while we succeeded in one property, we still have a lot to improve on in order to achieve the goal of a more accurate and faster tool.

Github repo link:

<https://github.com/kubaskibicki/CS453.git>

It is also on the title page.